

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل بیست و یکم: Link analysis

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

21.1 The Web as a graph

21.1.1 Anchor text and the web graph

21.2 PageRank

21.2.1 Markov chains

21.2.2 The PageRank computation

21.2.3 Topic-specific PageRank

21.3 Hubs and Authorities

21.3.1 Choosing the subset of the Web

20.5 Chapter 21 Summary

21.1 وب به عنوان یک گراف: قدرت نادیده گرفته لینکها

از ارجاعات علمی تا لینکهای وب 🔍

در دنیای آکادمیک، اهمیت یک مقاله علمی با تعداد دفعاتی که توسط مقالات دیگر به آن ارجاع داده می شود، سنجیده می شود. این ایده که «یک ارجاع، نوعی انتقال اعتبار است»، پایه و اساس یک حوزه به نام **Bibliometrics** است.

موتورهای جستجوی وب، به‌ویژه گوگل، این ایده را به دنیای وب آوردند. آنها فرض کردند که یک لینک از صفحه A به صفحه B، مانند یک ارجاع علمی، نوعی **تأیید اعتبار** برای صفحه B توسط سازنده صفحه A است. این ایده ساده اما قدرتمند، اساس الگوریتم‌های رتبه‌بندی مدرن را شکل داد.

قدرت متن لنگر: وقتی وب‌سایت شما خودش را نمی‌شناسد

اولین شهودی که از تحلیل لینک به دست می‌آید، قدرت **متن لنگر** است - متنی که در یک لینک قابل کلیک قرار می‌گیرد. اغلب، این متن توصیف دقیق‌تری از صفحه مقصد ارائه می‌دهد تا خود صفحه.

بیایید یک مثال واقعی را بررسی کنیم: صفحه اصلی شرکت Airbnb در گذشته، کلمه «اجاره آپارتمان» را در متن خود نداشت! این صفحه بیشتر بر روی «تجربه منحصر به فرد» و «میزبان‌های محلی» تمرکز داشت. اما وبلاگ‌های مسافرتی و رسانه‌ها به این صفحه با متن‌های لنگری مانند «اجاره خانه برای سفر» یا «پلتفرم اجاره اقامتگاه» لینک داده‌اند.

نتیجه؟ وقتی کاربران عبارت «اجاره آپارتمان» را جستجو می‌کردند، صفحه Airbnb در نتایج بالا ظاهر می‌شد - نه به خاطر محتوای صفحه، بلکه به خاطر توصیفی که جامعه وب از آن ارائه کرده بود. این نشان می‌دهد که چگونه **جامعه نویسندگان وب** به‌طور ناخواسته نقش یک سیستم ایندکس‌دهی جمعی را ایفا می‌کنند.

لینک به‌عنوان رأی اعتماد: چگونه گوگل «Big Blue» را کشف کرد

وقتی در گوگل عبارت «Big Blue» را جستجو می‌کنید، اولین نتیجه صفحه اصلی IBM است. چرا؟ چون هزاران وب‌سایت و وبلاگ با این متن لنگر به صفحه IBM لینک داده‌اند. گوگل از طریق تحلیل میلیون‌ها لینک، فهمید که «Big Blue» یک نام مستعار برای IBM است.

این مثال نشان می‌دهد که چگونه لینک‌ها می‌توانند ارتباطات معنایی را که در متن صفحات وجود ندارد، آشکار کنند. موتورهای جستجو این متن‌های لنگر را با وزن‌دهی هوشمند (مشابه مفهوم IDF در ایندکس‌گذاری) به واژگان ایندکس صفحه مقصد اضافه می‌کنند و معمولاً به آنها اهمیت بالاتری نسبت به متن خود صفحه می‌دهند.

محدودیت‌ها: چه زمانی یک لینک، رأی اعتماد نیست؟

همه لینک‌ها یکسان نیستند. موتورهای جستجو به سرعت فهمیدند که برخی لینک‌ها واقعاً رأی اعتماد نیستند. برای مثال:

- تقریباً تمام صفحات یک وب‌سایت دانشگاهی لینکی به صفحه «قوانین کپی‌رایت» یا «اطلاعات تماس با ما» دارند.
- آیا این یعنی صفحه «قوانین کپی‌رایت» مهم‌ترین صفحه آن وب‌سایت است؟ قطعاً خیر.

این نوع لینک‌ها که از یک **قالب مشترک** در تمام صفحات یک سایت ناشی می‌شوند، معمولاً توسط الگوریتم‌های تحلیل لینک نادیده گرفته یا وزن بسیار کمی به آنها اختصاص داده می‌شود، زیرا آنها نمایانگر یک تأیید واقعی و آگاهانه از محتوای صفحه مقصد نیستند.

🔗 اسپم متن لنگر: وقتی توصیف‌ها به دروغ تبدیل می‌شوند

قدرت متن لنگر همچنین زمینه‌ی سوءاستفاده را فراهم می‌کند. در گذشته، رقبا به صورت برنامه‌ریزی شده به صفحات یک شرکت با متن‌های لنگر منفی مانند «شرکت کلاهبرداری» لینک می‌دادند تا رتبه آن را در نتایج جستجو کاهش دهند.

این پدیده که به آن **اسپم متن لنگر** (Anchor Text Spam) گفته می‌شود، یکی از چالش‌های مهم موتورهای جستجو است. برای مقابله با آن، الگوریتم‌های پیچیده‌ای توسعه داده شده‌اند که الگوهای غیرطبیعی در توزیع متن‌های لنگر را شناسایی و وزن آنها را کاهش می‌دهند یا به طور کامل نادیده می‌گیرند.

📊 21.2 PageRank: چگونه گوگل اهمیت را در وب اندازه‌گیری می‌کند

🌐 یک جهانگرد در دنیای وب: ایده‌ی پشت PageRank

تصور کنید یک جهانگرد دیجیتال در حال گشت‌وگذار در اینترنت است. او از صفحه‌ای شروع کرده و در هر گام، یکی از لینک‌های موجود در صفحه را به صورت تصادفی انتخاب می‌کند و به آن صفحه می‌رود.

با این منطق، صفحاتی که جهانگرد بیشتر به آنها سر می‌زند، باید مهم‌تر باشند. چرا؟ چون احتمالاً صفحات مهم‌تر، لینک‌های بیشتری از صفحات دیگر دریافت کرده‌اند و جهانگرد را به سمت خود جذب می‌کنند. این ایده‌ی ساده اما ناب، اساس الگوریتم PageRank است.

🚫 دو چالش بزرگ: بن‌بست‌ها و جزیره‌های جداافتاده

اما جهانگرد ما با دو مشکل اساسی روبرو است:

1. **بن‌بست‌ها (Sink Nodes):** صفحه‌ای بدون هیچ لینک خروجی. جهانگرد به این صفحه می‌رسد و در آن گیر می‌کند.
2. **جزیره‌های جداافتاده:** ممکن است جهانگرد در یک بخش کوچک از وب (مثلاً مجموعه‌ای از سایت‌های یک دانشگاه خاص) گیر کند و هرگز به بخش‌های دیگر وب نرسد.

🌟 راه‌حل جادویی: عملیات تله‌پورت (Teleport)

برای حل این مشکلات، به جهانگرد ما یک توانایی جادویی می‌دهیم: **تله‌پورت**. در هر گام، جهانگرد با احتمال کوچکی (معمولاً ۰.۱ یا ۱۰٪) از صفحه فعلی «ناپدید» شده و به صورت کاملاً تصادفی به هر صفحه‌ای در وب

می‌رود.

این کار مانند این است که جهانگرد خسته از صفحه فعلی شده و یک آدرس جدید و تصادفی در نوار آدرس مرورگر خود تایپ کند. این مکانیزم ساده دو مشکل را همزمان حل می‌کند: هم از بن‌بست‌ها خارج می‌شود و هم به تمام بخش‌های وب دسترسی پیدا می‌کند.

PageRank در دنیای واقعی: داستان یک وبلاگ تکنولوژی

بیایید یک مثال واقعی را بررسی کنیم. دو وبلاگ تکنولوژی جدید، «تک‌بین» و «کدپارسی»، همزمان فعالیت خود را آغاز می‌کنند:

- **وبلاگ «تک‌بین»** موفق می‌شود از یک رسانه بزرگ معتبر مانند **وی‌جی‌بی‌آی** یک لینک دریافت کند.
- **وبلاگ «کدپارسی»** از ۵۰ وبلاگ کوچک‌تر اما تخصصی برنامه‌نویسی لینک می‌گیرد.

کدام وبلاگ احتمالاً رتبه بالاتری در گوگل خواهد داشت؟

الگوریتم PageRank به ما می‌گوید که لینک از **وی‌جی‌بی‌آی** (که خود PageRank بالایی دارد) بسیار ارزشمند است و به «تک‌بین» امتیاز قابل توجهی می‌دهد. اما در عین حال، مجموع امتیاز ۵۰ لینک از وبلاگ‌های کوچک‌تر نیز می‌تواند به همان اندازه یا حتی بیشتر باشد.

این مثال نشان می‌دهد که PageRank چگونه **اعتبار دیجیتال** را از طریق ساختار لینک‌های وب اندازه‌گیری می‌کند و به موتورهای جستجو کمک می‌کند تا صفحات معتبرتر را شناسایی کرده و در نتایج جستجو نمایش دهند.

In [2]: *PageRank پیاده‌سازی الگوریتم #*

```
import random

class PageRankSimulator:
    """
    با دو روش PageRank شبیه‌سازی الگوریتم
    1. PageRank پایه‌ی اصلی محاسبه - (Iterative) روش تکراری
    2. Random Surfer الگوریتم - شبیه‌سازی
    """

    def __init__(self, graph, alpha=0.1, tolerance=1e-6, max_iterations=100):
        """
        PageRank. مقداردهی اولیه شبیه‌ساز

        ورودی‌ها:
        - graph: دیکشنری نمایش‌دهنده گراف وب
          مثال: {"A": ["B", "C"], "B": ["C"], "C": ["A"]}
        - alpha: احتمال تلپورت (معمولاً 0.1)
        - tolerance: آستانه همگرایی برای روش تکراری
        - max_iterations: حداکثر تعداد تکرار برای جلوگیری از حلقه نامتناهی
        """
        self.graph = graph
        self.alpha = alpha
        self.tolerance = tolerance
        self.max_iterations = max_iterations

        # لیست تمام گره‌ها (صفحات)
        self.nodes = set(graph.keys())
        for neighbors in graph.values():
            self.nodes.update(neighbors)
        self.nodes = list(self.nodes)
        self.num_nodes = len(self.nodes)

        # تبدیل نام گره‌ها به اندیس برای سرعت بیشتر
        self.node_to_index = {node: i for i, node in enumerate(self.nodes)}

    def iterative_pagerank(self):
        """
        با روش تکراری (پایه‌ی موتورهای جستجوی واقعی) PageRank محاسبه
        """
```

```

فرمول اصلی:
PR(A) = (1 - α)/N + α * Σ(PR(Pi)/L(Pi))
که در آن:
- N = تعداد کل صفحات
- Pi = لینک داده‌اند A صفحاتی که به
- L(Pi) = تعداد لینک‌های خروجی صفحه
"""
N = self.num_nodes
# مقداره‌ی اولیه: همه صفحات امتیاز یکسان دارند
pagerank = [1.0 / N] * N
new_pagerank = [0.0] * N

for iteration in range(self.max_iterations):
    # برای همه صفحات (1-α)/N مرحله 1: محاسبه مقدار تله‌پورت
    teleport_value = (1.0 - self.alpha) / N
    new_pagerank = [teleport_value] * N

    # مرحله 2: توزیع امتیازات از طریق لینک‌ها
    for node in self.nodes:
        i = self.node_to_index[node]
        out_links = self.graph.get(node, [])
        if not out_links:
            continue # صفحه بدون لینک خروجی، امتیازی منتقل نمی‌کند

        # هر صفحه، امتیاز خود را به‌طور یکنواخت بین لینک‌های خروجی تقسیم می‌کند
        share = (self.alpha * pagerank[i]) / len(out_links)
        for neighbor in out_links:
            j = self.node_to_index[neighbor]
            new_pagerank[j] += share

    # بررسی همگرایی
    diff = sum(abs(new_pagerank[i] - pagerank[i]) for i in range(N))
    if diff < self.tolerance:
        print(f"✅ {iteration + 1} تکرار پس از همگرایی")
        break

    pagerank = new_pagerank[:]

# تبدیل نتیجه به دیکشنری خوانا
result = {}
for i, node in enumerate(self.nodes):
    result[node] = pagerank[i]
return result

def simulate_random_surfer(self, steps=100000):
    """
    PageRank برای درک شهودی Random Surfer شبیه‌سازی مستقیم.
    این روش برای آموزش مناسب است، اما در عمل کارآمد نیست.
    """
    # شروع از یک صفحه تصادفی
    current = random.choice(self.nodes)
    visit_count = {node: 0 for node in self.nodes}

    for _ in range(steps):
        visit_count[current] += 1

        # تصمیم‌گیری: تله‌پورت یا پیمایش لینک؟
        if random.random() < self.alpha:
            # تله‌پورت: رفتن به صفحه‌ای تصادفی
            current = random.choice(self.nodes)
        else:
            # پیمایش لینک: انتخاب یک لینک خروجی به‌صورت تصادفی
            out_links = self.graph.get(current, [])
            if out_links:
                current = random.choice(out_links)
            else:
                # اگر لینک خروجی نداشت، تله‌پورت اجباری
                current = random.choice(self.nodes)

    # تقریبی PageRank (تبدیل شمارش بازدیدها به احتمال)
    total_visits = sum(visit_count.values())
    approx_pagerank = {node: count / total_visits for node, count in visit_count.items()}
    return approx_pagerank

# مثال کاربردی: گراف نمونه از شکل 21.1
if __name__ == "__main__":
    # گراف نمونه: A → {B, C, D}, B → {C}, C → {A}, D → {}
    sample_graph = {
        "A": ["B", "C", "D"],
        "B": ["C"],
        "C": ["A"],
        "D": [] # صفحه بدون لینک خروجی (Sink Node)
    }

    print("برای گراف نمونه PageRank مثال آموزشی")
    print("-" * 50)
    print("ساختار گراف:")
    for node, links in sample_graph.items():
        print(f" {node} → {links if links else '[] (بدون لینک خروجی)'}")

    # با روش تکراری PageRank محاسبه
    pr_simulator = PageRankSimulator(sample_graph, alpha=0.1)
    iterative_result = pr_simulator.iterative_pagerank()

    print(f"\n📊 PageRank (روش تکراری) نتایج:")
    sorted_pages = sorted(iterative_result.items(), key=lambda x: x[1], reverse=True)
    for rank, (page, score) in enumerate(sorted_pages, 1):

```

```
print(f" {rank}. صفحه {page}: {score:.6f}")

# را بالا ببرید steps برای همگرایی، مقدار Random Surfer تأیید با شبیه‌سازی
print("\n 🎲 Random Surfer (100,000 گام):")
surfer_result = pr_simulator.simulate_random_surfer(steps=100000)
surfer_sorted = sorted(surfer_result.items(), key=lambda x: x[1], reverse=True)
for rank, (page, score) in enumerate(surfer_sorted, 1):
    print(f" {rank}. صفحه {page}: {score:.6f}")

print("\n 💡 نکته:")
print("- لینک دریافت می‌کند B و هم از A بالاترین امتیاز را دارد چون هم از 'C' صفحه -")
print("- لینک دریافت می‌کند A امتیاز پایینی دارد چون لینک خروجی ندارد و فقط از 'D' صفحه -")
print("- جلوگیری می‌کند 'D' از بن‌بست صفحه (تله‌پورت) α پارامتر -")
```

برای گراف نمونه PageRank مثال آموزشی

ساختار گراف:

- A → ['B', 'C', 'D']
- B → ['C']
- C → ['A']
- D → [] (بدون لینک خروجی)

همگرایی پس از 5 تکرار ✓

📊 نتایج PageRank (روش تکراری):

- صفحه C: 0.256692
- صفحه A: 0.250669
- صفحه B: 0.233356
- صفحه D: 0.233356

🎲 Random Surfer (100,000 گام):

- صفحه A: 0.346840
- صفحه C: 0.318120
- صفحه D: 0.168400
- صفحه B: 0.166640

💡 نکته:

- لینک دریافت می‌کند B و هم از A بالاترین امتیاز را دارد چون هم از 'C' صفحه -
- لینک دریافت می‌کند A امتیاز پایینی دارد چون لینک خروجی ندارد و فقط از 'D' صفحه -
- جلوگیری می‌کند 'D' از بن‌بست صفحه (تله‌پورت) α پارامتر -

21.2.1 زنجیره‌های مارکوف: سفر تصادفی در وب 🌐

🏠 زنجیره مارکوف: هر قدم فقط به جای فعلی بستگی دارد

تصور کنید در یک شهر جدید هستید و از یک توریست بپرسید: "اگر من الان در میدان اصلی هستم، کجا احتمالاً بعدی است؟" او به شما لیستی از مکان‌ها و احتمال رفتن به هر کدام را می‌دهد. سپس از میدان اصلی به یکی از آن مکان‌ها می‌روید و دوباره از یک توریست در آنجا می‌پرسید: "اگر من الان اینجا هستم، کجا بعدی می‌روم؟"

این یک **زنجیره مارکوف** است! در دنیای وب، زنجیره مارکوف یک مدل ریاضی برای توصیف حرکت یک کاربر بین صفحات وب است. ویژگی کلیدی این است که **آینده فقط به حالت فعلی بستگی دارد، نه به گذشته**. این یعنی اگر شما در صفحه A هستید، احتمال رفتن شما به صفحه B فقط به صفحه فعلی (A) بستگی دارد، نه به صفحاتی که قبلاً بازدید کرده‌اید.

این ویژگی که به آن **خاصیت مارکوف** می‌گویند، به ما اجازه می‌دهد تا رفتار کاربران در وب را با یک مدل ساده اما قدرتمند توصیف کنیم.

🔍 ماتریس انتقال احتمال: نقشه احتمالات وب

در زنجیره مارکوف، از یک **ماتریس انتقال احتمال** برای نمایش احتمال حرکت از یک صفحه به صفحه دیگر استفاده می‌شود. این ماتریس به شکل $N \times N$ است که N تعداد صفحات وب است.

هر درایه P_{ij} در این ماتریس نشان‌دهنده احتمال حرکت از صفحه i به صفحه j است. ویژگی‌های این ماتریس عبارتند از:

- هر درایه P_{ij} در بازه $[0, 1]$ قرار دارد.
- مجموع درایه‌های هر سطر برابر با 1 است ($\sum P_{ij} = 1$).
- چنین ماتریسی یک **ماتریس تصادفی** (Stochastic Matrix) نامیده می‌شود.

برای درک بهتر، به مثال زیر توجه کنید:

فرض کنید سه سایت خبری A، B و C داریم: - سایت A به سایت‌های B و C لینک دارد. - سایت B فقط به سایت A لینک دارد. - سایت C فقط به سایت A لینک دارد.

ماتریس انتقال احتمال برای این شبکه به شکل زیر خواهد بود:

$$A \begin{bmatrix} 0 & 0.5 & 0.5 \end{bmatrix} \quad B \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \quad C \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

این ماتریس نشان می‌دهد که اگر در سایت A باشیم، 50% احتمال رفتن به B و 50% احتمال رفتن به C داریم. اما اگر در B یا C باشیم، با احتمال 100% به A بازمی‌گردیم.

🌐 ساخت ماتریس انتقال از گراف وب

برای ساخت ماتریس انتقال P از گراف وب، مراحل زیر را طی می‌کنیم:

1. ابتدا **ماتریس مجاورت A** را می‌سازیم: اگر از صفحه i به صفحه j لینک باشد، $A_{ij} = 1$ ، در غیر این صورت $A_{ij} = 0$.
2. اگر صفحه‌ای لینک خروجی نداشته باشد (صفحه تله)، تمام درایه‌های سطر آن برابر $N/1$ می‌شود (تله‌پورت اجباری).
3. در سایر صفحات، هر لینک خروجی احتمال $1/(\text{تعداد لینک‌ها})$ دریافت می‌کند.
4. کل ماتریس در $(1-\alpha)$ ضرب می‌شود.
5. مقدار α/N به هر درایه اضافه می‌شود تا تله‌پورت جهانی را مدل کند.

پارامتر α (معمولاً بین 0.1 تا 0.2) احتمال **تله‌پورت** یا پرش تصادفی کاربر به هر صفحه وب را نشان می‌دهد، حتی اگر لینکی به آن صفحه وجود نداشته باشد. این ویژگی مدل را واقعی‌تر می‌کند، زیرا کاربران واقعی همیشه از طریق لینک‌ها حرکت نمی‌کنند و ممکن است مستقیماً آدرس صفحه‌ای را در نوار آدرس تایپ کنند.

به عبارت دیگر، تله‌پورت به کاربر اجازه می‌دهد که در هر لحظه با احتمال α به هر صفحه‌ای در وب (حتی صفحاتی که به آن لینک ندارد) پرش کند. این ویژگی باعث می‌شود که زنجیره مارکوف **ارگودیک** شود و تضمین کند که حالت پایدار وجود دارد.

🇮🇷 توزیع احتمال و بردار احتمال: کجا هستیم؟

در هر لحظه، موقعیت کاربر در وب را می‌توان با یک **بردار احتمال** نمایش داد. این بردار به ما می‌گوید که کاربر با چه احتمالی در هر صفحه قرار دارد.

اگر در زمان $t=0$ کاربر در صفحه i باشد، بردار احتمال اولیه به این صورت است: درایه i برابر 1 و بقیه درایه‌ها برابر 0. پس از یک گام ($t=1$)، توزیع احتمال جدید با ضرب بردار احتمال فعلی در ماتریس انتقال به دست می‌آید: $x_1 \sim x_0 P$.

به طور کلی، پس از t گام، توزیع احتمال برابر است با $x_t = \sim x_0 P^t$.

این فرآیند را می‌توان به این صورت تصور کرد: در هر گام، بردار احتمال فعلی (که نشان می‌دهد کاربر با چه احتمالی در هر صفحه است) با ماتریس انتقال ضرب می‌شود تا بردار احتمال جدید به دست آید.

🚩 حالت پایدار و PageRank: مقصد نهایی سفر تصادفی

اگر زنجیره مارکوف ارگودیک باشد (یعنی هم غیرقابل تجزیه و هم نادره‌ای باشد)، پس از تعداد زیادی گام، توزیع احتمال به یک حالت پایدار همگرا می‌شود. در این حالت، احتمال حضور در هر صفحه دیگر تغییر نمی‌کند.

این حالت پایدار را می‌توان با بردار π نمایش داد که معادله زیر را برآورده می‌کند:

$$\pi P = \pi$$

این بردار پایدار، بردار ویسته چپ اصلی ماتریس P است که به بزرگترین مقدار ویژه (که برابر 1 است) تعلق دارد.

حالا نکته کلیدی: **PageRank هر صفحه برابر با احتمال حضور در آن صفحه در حالت پایدار است!** به عبارت دیگر، PageRank صفحه i برابر با $\pi(i)$ است.

این تعریف زیبا به ما می‌گوید که PageRank یک صفحه، احتمال حضور یک کاربر تصادفی در آن صفحه پس از گشت‌وگذار طولانی در وب است. صفحاتی که PageRank بالاتری دارند، صفحاتی هستند که کاربران تصادفی بیشتر به آن‌ها سر می‌زنند.

قضیه 21.1 این را به صورت رسمی بیان می‌کند: برای هر زنجیره مارکوف ارگودیک، یک بردار احتمال پایدار منحصر به فرد π وجود دارد که بردار ویسته چپ اصلی P است. اگر $\eta(i, t)$ تعداد بازدیدها از حالت i در t گام باشد، آنگاه:

$$\lim_{t \rightarrow \infty} \eta(i, t) / t = \pi(i)$$

که در آن $\pi(i)$ احتمال پایدار برای حالت i است و $\pi(i) > 0$ است.

🏠 مثال واقعی: PageRank در وبسایت‌های خبری

تصور کنید یک شبکه کوچک از وبسایت‌های خبری داریم:

- **خبرگزاری الف:** یک وبسایت خبری معتبر که به خبرگزاری ب و ج لینک داده است.
- **خبرگزاری ب:** یک وبسایت متوسط که فقط به خبرگزاری الف لینک داده است.
- **خبرگزاری ج:** یک وبسایت کوچک که به خبرگزاری الف و ب لینک داده است.
- **وبلاگ د:** یک وبلاگ شخصی که فقط به خبرگزاری الف لینک داده است.

با فرض $\alpha=0.15$ (احتمال تله‌پورت)، ماتریس انتقال برای این شبکه به شکل زیر خواهد بود:

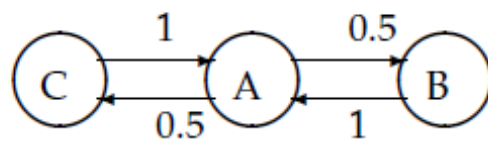
الف ب ج د الف [0.0375 0.4625 0.4625 0.0375] ب [0.0375 0.0375 0.0375 0.8875] ج [0.0375 0.0375 0.0375 0.8875] د [0.0375 0.0375 0.4625 0.4625]

پس از محاسبه حالت پایدار، PageRank این وب‌سایت‌ها به شکل زیر خواهد بود:

خبرگزاری الف: 0.387 خبرگزاری ب: 0.246 خبرگزاری ج: 0.215 وبلاگ د: 0.152

نتایج نشان می‌دهد که خبرگزاری الف بالاترین PageRank را دارد، زیرا هر سه وب‌سایت دیگر به آن لینک داده‌اند. این مثال ساده نشان می‌دهد که چگونه PageRank اهمیت یک صفحه را بر اساس تعداد و کیفیت لینک‌های ورودی به آن ارزیابی می‌کند.

جالب است بدانید که الگوریتم PageRank که توسط لری پیج و سرگی برین توسعه داده شد، اساس موفقیت اولیه گوگل بود و به گوگل کمک کرد تا نتایج جستجوی باکیفیت‌تری نسبت به رقبای خود ارائه دهد.



► Figure 21.2 A simple Markov chain with three states; the numbers on the links indicate the transition probabilities.

شکل 21.2: یک زنجیره مارکوف ساده با سه حالت. اعداد روی لینک‌ها احتمال انتقال را نشان می‌دهند.

21.2.2 محاسبه PageRank: از تئوری تا عمل

🎯 چگونه PageRank محاسبه می‌شود؟

PageRank هر صفحه برابر است با مقدار ورودی متناظر در بردار ویژه چپ اصلی ماتریس انتقال احتمال P است. این بردار π در معادله زیر صدق می‌کند:

$$\pi P = \lambda \pi$$

که در آن λ یک مقدار ویژه است. برای توزیع ایستا، $\lambda = 1$ است، بنابراین:

$$\pi P = \pi$$

📖 روش تکرار توان: شبیه‌سازی حرکت کاربر در وب

برای محاسبه این بردار، از روشی به نام **تکرار توان** (Power Iteration) استفاده می‌شود. این روش را می‌توان به این صورت تصور کرد:

تصور کنید یک کاربر اینترنت هستید که به صورت تصادفی روی لینک‌ها کلیک می‌کند. از صفحه‌ای شروع می‌کنید و سپس لینک‌های تصادفی را دنبال می‌کنید. گاهی هم (با احتمال α) به صورت تصادفی به هر صفحه‌ای در اینترنت می‌پرید (تله‌پورت). اگر این کار را برای مدت طولانی ادامه دهید، در نهایت متوجه می‌شوید که درصد زمانی که در هر صفحه می‌گذرانید به یک مقدار ثابت همگرا می‌شود. این درصد زمانی همان PageRank آن صفحه است!

از نظر ریاضی، اگر $x \sim$ توزیع اولیه احتمال روی حالت‌ها باشد، آنگاه توزیع در زمان t برابر است با $x P^t$. با افزایش t ، این توزیع به یک **توزیع ایستا** همگرا می‌شود که مستقل از توزیع اولیه است (طبق قضیه 21.1).

مثال عملی: شبکه کوچک فروشگاه‌های آنلاین

فرض کنید سه فروشگاه آنلاین داریم: فروشگاه اصلی (A) و دو فروشگاه وابسته (B و C). فروشگاه اصلی به هر دو فروشگاه وابسته لینک دارد، و هر دو فروشگاه وابسته فقط به فروشگاه اصلی لینک دارند. با نرخ تله‌پورت $\alpha = 0.5$ ، ماتریس انتقال احتمال به صورت زیر است:

$$P = \begin{pmatrix} 1 & 1/6 & 2/3 & 1/6 \\ 5/12 & 1/6 & 5/12 & 1/6 \\ 2/3 & 1/6 & 5/12 & 1/6 \end{pmatrix}$$

اگر کاربر از فروشگاه اصلی (A) شروع کند (یعنی $x_0 = (1, 0, 0)$)، توزیع احتمال در هر گام به صورت زیر محاسبه می‌شود:

گام	$x_t \sim$ توزیع احتمال
0	(1, 0, 0)
1	(1/6, 2/3, 1/6)
2	(1/3, 1/3, 1/3)
3	(1/4, 1/2, 1/4)
4	(7/24, 5/12, 7/24)
∞	(5/18, 4/9, 5/18)

با گذشت زمان، توزیع به مقدار ایستای $\pi = (5/18, 4/9, 5/18)$ همگرا می‌شود. این یعنی کاربر در حالت پایدار حدود 28% زمان خود را در فروشگاه اصلی و 36% زمان را در هر یک از فروشگاه‌های وابسته می‌گذراند.

در این مثال ساده، می‌توان توزیع ایستا را مستقیماً با استفاده از تقارن گراف محاسبه کرد: حالت‌های 1 و 3 متقارن هستند (چون سطرهای اول و سوم ماتریس P یکسان‌اند). بنابراین، اگر احتمال ایستای هر یک از این دو حالت را p فرض کنیم، آنگاه توزیع ایستا به صورت $\pi = (p, 1 - 2p, p)$ خواهد بود. با جایگذاری در معادله $\pi P = \pi$ ، معادله ساده‌ای به دست می‌آید که راه‌حل آن $p = 5/18$ است.

مثال پیچیده‌تر: شبکه وبلاگ‌های فناوری

در مثال 21.1، یک گراف کوچک از وبلاگ‌های فناوری را در نظر بگیرید (شکل 21.4). این وبلاگ‌ها در مورد موضوعات مختلفی مانند خودرو، حیوانات و سرعت نوشته‌اند و به هم لینک داده‌اند. با نرخ تله‌پورت 0.14،

ماتریس انتقال احتمال به صورت زیر است:

```
0.02 0.02 0.88 0.02 0.02 0.02 0.02
0.02 0.45 0.45 0.02 0.02 0.02 0.02
0.31 0.02 0.31 0.31 0.02 0.02 0.02
0.02 0.02 0.02 0.45 0.45 0.02 0.02
0.02 0.02 0.02 0.02 0.02 0.02 0.88
0.02 0.02 0.02 0.02 0.02 0.45 0.45
0.02 0.02 0.02 0.31 0.31 0.02 0.31
```

و بردار PageRank متناظر آن عبارت است از:

$\sim x = (0.05, 0.04, 0.11, 0.25, 0.21, 0.04, 0.31)$

در این شبکه، وبلاگ‌های q4، q3، q2 و q6 هر کدام حداقل دو لینک ورودی دارند. از میان این‌ها، q2 کمترین PageRank را دارد، چون پیمایش تصادفی تمایل دارد از بخش بالایی گراف خارج شود و تنها از طریق تله‌پورت بتواند به آن بازگردد. وبلاگ q7 بالاترین PageRank را دارد زیرا از بخش‌های مختلف گراف لینک‌های ورودی دریافت می‌کند.

PageRank در موتورهای جستجوی واقعی 🔍

مقادیر PageRank صفحات (و رتبه‌بندی ضمنی آن‌ها) **مستقل از هر پرس‌وجویی** است که کاربر ممکن است مطرح کند؛ بنابراین، PageRank یک معیار ایستا و مستقل از پرس‌وجو برای کیفیت ایستای هر صفحه وب است.

اما در موتورهای جستجوی واقعی مانند گوگل، PageRank فقط یکی از صدها عامل در رتبه‌بندی نتایج جستجو است. برای مثال، وقتی شما عبارت "خرید لپ‌تاپ" را جستجو می‌کنید، گوگل نه تنها PageRank صفحات را در نظر می‌گیرد، بلکه به عواملی مانند:

- مرتبط بودن محتوای صفحه با عبارت جستجو
- کیفیت و اعتبار صفحه
- تجربه کاربری صفحه (سرعت بارگذاری، طراحی ریسپانسیو و ...)
- موقعیت جغرافیایی کاربر
- تاریخچه جستجوی کاربر

نیز توجه می‌کند. در واقع، سهم نسبی PageRank در امتیاز کلی ممکن است توسط الگوریتم‌های یادگیری ماشین تعیین شود که با تحلیل میلیاردها جستجو، بهترین ترکیب از عوامل را برای هر نوع پرس‌وجو مشخص می‌کنند.

این رویکرد ترکیبی به گوگل اجازه می‌دهد تا نه تنها صفحات با اعتبار عمومی بالا را نمایش دهد، بلکه نتایجی را ارائه کند که دقیقاً به نیاز کاربر پاسخ می‌دهند.

In [4]: # *PageRank برای محاسبه Power Iteration پیاده‌سازی روش*

```
def power_iteration_pagerank(transition_matrix, initial_vector=None, tolerance=1e-6, max_iterations=100):
    """
    Power Iteration محاسبه بردار ویژه چپ اصلی (توزیع ایستای) با روش
    ورودی‌ها:
    - transition_matrix: ماتریس انتقال احتمال
    - initial_vector: بردار اولیه توزیع احتمال (پیش‌فرض: توزیع یکنواخت)
    - tolerance: آستانه همگرایی
    - max_iterations: حداکثر تعداد تکرار

    خروجی:
    صفحات را نشان می‌دهد PageRank که  $\pi$  بردار ایستای -
    """
    N = len(transition_matrix)

    # مقداردهی اولیه: توزیع یکنواخت اگر بردار اولیه داده نشده باشد
    if initial_vector is None:
        x = [1.0 / N] * N
    else:
        x = initial_vector[:]

    print("PageRank برای محاسبه Power Iteration شروع روش")
    print(f"{'>6:'} | {'بردار توزیع احتمال'} | {'تکرار'}")
    print("-" * 55)
    print(f"{'<6:'} | {[f'{v:.6f}' for v in x]}")

    for iteration in range(1, max_iterations + 1):
        # محاسبه  $x_{new} = x * P$ 
        x_new = [0.0] * N
        for j in range(N): # برای هر ستون j
            for i in range(N): # برای هر سطر i
                x_new[j] += x[i] * transition_matrix[i][j]

        #  $x$  و  $x_{new}$  بررسی همگرایی: تفاوت بین
        diff = sum(abs(x_new[i] - x[i]) for i in range(N))
        print(f"iteration:<6} | {[f'{v:.6f}' for v in x_new]}")

        if diff < tolerance:
            print(f"\n✅ تفاوت = {diff:.2e} {iteration} تکرار همگرایی پس از")
            return x_new

        x = x_new[:]

    print(f"\n⚠️ {max_iterations} هشدار: همگرایی پس از")
    return x

# مثال 1: ماتریس انتقال از شکل 21.3 (مثال کتاب)
if __name__ == "__main__":
    print("برای ماتریس انتقال شکل 21.3 PageRank مثال 1: محاسبه")
    print("=" * 60)

    # از فرمول (21.3)  $P$  ماتریس انتقال
    P_example = [
        [1/6, 2/3, 1/6], # سطر اول
        [5/12, 1/6, 5/12], # سطر دوم
        [1/6, 2/3, 1/6] # سطر سوم
    ]

    #  $x_0 = [1, 0, 0]$  بردار اولیه: شروع از حالت اول
    x0 = [1.0, 0.0, 0.0]

    result = power_iteration_pagerank(P_example, initial_vector=x0, tolerance=1e-8)

    print(f"\n{'<6:'} | {'توزیع ایستای نهایی'} | {'result'}")
    print(f"بررسی: {'<6:'} | {'result[0]:.6f'} ≈ 5/18 = {5/18:.6f}, {'result[1]:.6f'} ≈ 4/9 = {4/9:.6f}")

    print("\n" + "=" * 60)
    print("برای گراف شکل 21.4 PageRank مثال 2: محاسبه")
    print("=" * 60)

    # 21.4 (با  $\alpha=0.14$ ) ماتریس انتقال برای گراف 7 گره‌ای شکل
    P_fig21_4 = [
        [0.02, 0.02, 0.88, 0.02, 0.02, 0.02, 0.02],
        [0.02, 0.45, 0.45, 0.02, 0.02, 0.02, 0.02],
        [0.31, 0.02, 0.31, 0.31, 0.02, 0.02, 0.02],
        [0.02, 0.02, 0.02, 0.45, 0.45, 0.02, 0.02],
        [0.02, 0.02, 0.02, 0.02, 0.02, 0.02, 0.88],
        [0.02, 0.02, 0.02, 0.02, 0.02, 0.45, 0.45],
        [0.02, 0.02, 0.02, 0.31, 0.31, 0.02, 0.31]
    ]

    result2 = power_iteration_pagerank(P_fig21_4, tolerance=1e-8)
    print(f"\nبردار PageRank: {[f'{v:.6f}' for v in result2]}")
    print("بالاترین امتیاز را دارد، همان‌طور که در متن کتاب توضیح داده شده (ایندکس 6) d6 نکته: صفحه")
```

```
print("\n💡 نکته:")
print("- را بدون جبر خطی پیچیده محاسبه کرد PageRank این پیاده‌سازی نشان می‌دهد که چگونه می‌توان -")
print("- در موتورهای جستجو است PageRank پایهی الگوریتم‌های واقعی محاسبه، Power Iteration روش -")
print("- بستگی دارد، نه به نقطه شروع  $\alpha$  توزیع ایستا صرفاً به ساختار گراف و مقدار -")
```

برای ماتریس انتقال شکل PageRank 21.3 مثال 1: محاسبه

=====

PageRank... برای محاسبه Power Iteration شروع روش

تکرار | بردار توزیع احتمال

0		['1.000000', '0.000000', '0.000000']
1		['0.166667', '0.666667', '0.166667']
2		['0.333333', '0.333333', '0.333333']
3		['0.250000', '0.500000', '0.250000']
4		['0.291667', '0.416667', '0.291667']
5		['0.270833', '0.458333', '0.270833']
6		['0.281250', '0.437500', '0.281250']
7		['0.276042', '0.447917', '0.276042']
8		['0.278646', '0.442708', '0.278646']
9		['0.277344', '0.445312', '0.277344']
10		['0.277995', '0.444010', '0.277995']
11		['0.277669', '0.444661', '0.277669']
12		['0.277832', '0.444336', '0.277832']
13		['0.277751', '0.444499', '0.277751']
14		['0.277791', '0.444417', '0.277791']
15		['0.277771', '0.444458', '0.277771']
16		['0.277781', '0.444438', '0.277781']
17		['0.277776', '0.444448', '0.277776']
18		['0.277779', '0.444443', '0.277779']
19		['0.277777', '0.444445', '0.277777']
20		['0.277778', '0.444444', '0.277778']
21		['0.277778', '0.444445', '0.277778']
22		['0.277778', '0.444444', '0.277778']
23		['0.277778', '0.444444', '0.277778']
24		['0.277778', '0.444444', '0.277778']
25		['0.277778', '0.444444', '0.277778']
26		['0.277778', '0.444444', '0.277778']
27		['0.277778', '0.444444', '0.277778']
28		['0.277778', '0.444444', '0.277778']

همگرایی پس از 28 تکرار (e-09 تفاوت = 9.93) ✓

توزیع ایستای نهایی: [0.2777777860562003, 0.4444444427887597, 0.27777777860562003]
بررسی: 0.444444 = 4/9 ≈ 0.444444, 0.277778 = 5/18 ≈ 0.277778

=====

برای گراف شکل PageRank 21.4 مثال 2: محاسبه

=====

PageRank... برای محاسبه Power Iteration شروع روش

تکرار | بردار توزیع احتمال

0		['0.142857', '0.142857', '0.142857', '0.142857', '0.142857', '0.142857', '0.142857']
1		['0.061429', '0.081429', '0.245714', '0.164286', '0.122857', '0.081429', '0.245714']
2		['0.091314', '0.055071', '0.179157', '0.233214', '0.161957', '0.055071', '0.231986']
3		['0.072111', '0.043836', '0.174322', '0.239669', '0.187713', '0.043836', '0.250395']
4		['0.070791', '0.039087', '0.151656', '0.246463', '0.195910', '0.039087', '0.273135']
5		['0.064303', '0.037130', '0.141991', '0.249491', '0.205511', '0.037130', '0.284822']
6		['0.061585', '0.036374', '0.132851', '0.251464', '0.210287', '0.036374', '0.295711']
7		['0.059020', '0.036134', '0.127623', '0.252906', '0.214379', '0.036134', '0.302737']
8		['0.057589', '0.036116', '0.123884', '0.254133', '0.217122', '0.036116', '0.308276']
9		['0.056591', '0.036195', '0.121648', '0.255268', '0.219342', '0.036195', '0.312319']
10		['0.056029', '0.036315', '0.120261', '0.256367', '0.221089', '0.036315', '0.315521']
11		['0.055714', '0.036453', '0.119514', '0.257452', '0.222577', '0.036453', '0.318091']
12		['0.055584', '0.036600', '0.119173', '0.258535', '0.223876', '0.036600', '0.320262']
13		['0.055573', '0.036751', '0.119113', '0.259619', '0.225059', '0.036751', '0.322160']
14		['0.055643', '0.036903', '0.119239', '0.260706', '0.226163', '0.036903', '0.323880']
15		['0.055768', '0.037057', '0.119490', '0.261797', '0.227218', '0.037057', '0.325483']
16		['0.055929', '0.037212', '0.119824', '0.262892', '0.228240', '0.037212', '0.327009']
17		['0.056115', '0.037368', '0.120216', '0.263992', '0.229242', '0.037368', '0.328486']
18		['0.056318', '0.037524', '0.120646', '0.265096', '0.230233', '0.037524', '0.329933']
19		['0.056533', '0.037681', '0.121102', '0.266205', '0.231217', '0.037681', '0.331362']
20		['0.056755', '0.037838', '0.121576', '0.267318', '0.232199', '0.037838', '0.332780']
21		['0.056983', '0.037997', '0.122063', '0.268436', '0.233179', '0.037997', '0.334194']
22		['0.057215', '0.038155', '0.122559', '0.269559', '0.234161', '0.038155', '0.335606']
23		['0.057450', '0.038315', '0.123062', '0.270686', '0.235144', '0.038315', '0.337019']
24		['0.057688', '0.038475', '0.123571', '0.271818', '0.236130', '0.038475', '0.338435']
25		['0.057927', '0.038636', '0.124083', '0.272955', '0.237120', '0.038636', '0.339854']
26		['0.058168', '0.038798', '0.124600', '0.274097', '0.238113', '0.038798', '0.341279']
27		['0.058411', '0.038960', '0.125119', '0.275243', '0.239110', '0.038960', '0.342708']
28		['0.058655', '0.039123', '0.125641', '0.276395', '0.240110', '0.039123', '0.344143']
29		['0.058900', '0.039287', '0.126166', '0.277551', '0.241115', '0.039287', '0.345583']
30		['0.059146', '0.039451', '0.126693', '0.278712', '0.242124', '0.039451', '0.347029']
31		['0.059393', '0.039616', '0.127222', '0.279877', '0.243136', '0.039616', '0.348481']
32		['0.059641', '0.039782', '0.127754', '0.281048', '0.244153', '0.039782', '0.349938']
33		['0.059891', '0.039948', '0.128288', '0.282223', '0.245175', '0.039948', '0.351402']
34		['0.060141', '0.040115', '0.128825', '0.283404', '0.246200', '0.040115', '0.352872']
35		['0.060393', '0.040283', '0.129364', '0.284589', '0.247230', '0.040283', '0.354348']
36		['0.060645', '0.040451', '0.129905', '0.285780', '0.248264', '0.040451', '0.355830']
37		['0.060899', '0.040621', '0.130448', '0.286975', '0.249303', '0.040621', '0.357319']
38		['0.061154', '0.040791', '0.130993', '0.288175', '0.250345', '0.040791', '0.358813']
39		['0.061409', '0.040961', '0.131541', '0.289380', '0.251392', '0.040961', '0.360314']
40		['0.061666', '0.041133', '0.132092', '0.290591', '0.252444', '0.041133', '0.361821']
41		['0.061924', '0.041305', '0.132644', '0.291806', '0.253500', '0.041305', '0.363334']
42		['0.062183', '0.041477', '0.133199', '0.293027', '0.254560', '0.041477', '0.364854']
43		['0.062443', '0.041651', '0.133756', '0.294252', '0.255625', '0.041651', '0.366380']
44		['0.062704', '0.041825', '0.134315', '0.295483', '0.256694', '0.041825', '0.367912']
45		['0.062967', '0.042000', '0.134877', '0.296719', '0.257768', '0.042000', '0.369451']
46		['0.063230', '0.042176', '0.135441', '0.297960', '0.258846', '0.042176', '0.370997']
47		['0.063494', '0.042352', '0.136008', '0.299206', '0.259928', '0.042352', '0.372548']
48		['0.063760', '0.042529', '0.136577', '0.300458', '0.261015', '0.042529', '0.374106']
49		['0.064027', '0.042707', '0.137148', '0.301714', '0.262107', '0.042707', '0.375671']
50		['0.064295', '0.042886', '0.137722', '0.302976', '0.263203', '0.042886', '0.377243']

51 | ['0.064563', '0.043065', '0.138298', '0.304244', '0.264304', '0.043065', '0.378820']
52 | ['0.064833', '0.043245', '0.138876', '0.305516', '0.265410', '0.043245', '0.380405']
53 | ['0.065105', '0.043426', '0.139457', '0.306794', '0.266520', '0.043426', '0.381996']
54 | ['0.065377', '0.043608', '0.140040', '0.308077', '0.267635', '0.043608', '0.383594']
55 | ['0.065650', '0.043790', '0.140626', '0.309366', '0.268754', '0.043790', '0.385198']
56 | ['0.065925', '0.043973', '0.141214', '0.310660', '0.269878', '0.043973', '0.386809']
57 | ['0.066201', '0.044157', '0.141805', '0.311959', '0.271007', '0.044157', '0.388427']
58 | ['0.066478', '0.044342', '0.142398', '0.313264', '0.272140', '0.044342', '0.390052']
59 | ['0.066756', '0.044527', '0.142993', '0.314574', '0.273279', '0.044527', '0.391683']
60 | ['0.067035', '0.044714', '0.143591', '0.315890', '0.274422', '0.044714', '0.393321']
61 | ['0.067315', '0.044901', '0.144192', '0.317211', '0.275570', '0.044901', '0.394966']
62 | ['0.067597', '0.045088', '0.144795', '0.318538', '0.276722', '0.045088', '0.396618']
63 | ['0.067880', '0.045277', '0.145401', '0.319870', '0.277880', '0.045277', '0.398277']
64 | ['0.068163', '0.045466', '0.146009', '0.321208', '0.279042', '0.045466', '0.399943']
65 | ['0.068449', '0.045656', '0.146620', '0.322551', '0.280209', '0.045656', '0.401616']
66 | ['0.068735', '0.045847', '0.147233', '0.323901', '0.281381', '0.045847', '0.403296']
67 | ['0.069022', '0.046039', '0.147849', '0.325255', '0.282558', '0.046039', '0.404983']
68 | ['0.069311', '0.046232', '0.148467', '0.326616', '0.283740', '0.046232', '0.406676']
69 | ['0.069601', '0.046425', '0.149088', '0.327982', '0.284926', '0.046425', '0.408377']
70 | ['0.069892', '0.046619', '0.149712', '0.329354', '0.286118', '0.046619', '0.410085']
71 | ['0.070184', '0.046814', '0.150338', '0.330731', '0.287315', '0.046814', '0.411801']
72 | ['0.070478', '0.047010', '0.150967', '0.332115', '0.288517', '0.047010', '0.413523']
73 | ['0.070773', '0.047207', '0.151598', '0.333504', '0.289723', '0.047207', '0.415253']
74 | ['0.071069', '0.047404', '0.152232', '0.334899', '0.290935', '0.047404', '0.416989']
75 | ['0.071366', '0.047602', '0.152869', '0.336299', '0.292152', '0.047602', '0.418734']
76 | ['0.071664', '0.047802', '0.153508', '0.337706', '0.293374', '0.047802', '0.420485']
77 | ['0.071964', '0.048001', '0.154150', '0.339118', '0.294601', '0.048001', '0.422244']
78 | ['0.072265', '0.048202', '0.154795', '0.340537', '0.295833', '0.048202', '0.424010']
79 | ['0.072567', '0.048404', '0.155442', '0.341961', '0.297071', '0.048404', '0.425783']
80 | ['0.072871', '0.048606', '0.156093', '0.343391', '0.298313', '0.048606', '0.427564']
81 | ['0.073176', '0.048810', '0.156745', '0.344828', '0.299561', '0.048810', '0.429352']
82 | ['0.073482', '0.049014', '0.157401', '0.346270', '0.300814', '0.049014', '0.431148']
83 | ['0.073789', '0.049219', '0.158059', '0.347718', '0.302072', '0.049219', '0.432951']
84 | ['0.074098', '0.049425', '0.158721', '0.349173', '0.303335', '0.049425', '0.434762']
85 | ['0.074408', '0.049631', '0.159384', '0.350633', '0.304604', '0.049631', '0.436581']
86 | ['0.074719', '0.049839', '0.160051', '0.352100', '0.305878', '0.049839', '0.438407']
87 | ['0.075031', '0.050047', '0.160720', '0.353572', '0.307157', '0.050047', '0.440240']
88 | ['0.075345', '0.050257', '0.161393', '0.355051', '0.308442', '0.050257', '0.442082']
89 | ['0.075660', '0.050467', '0.162068', '0.356536', '0.309732', '0.050467', '0.443931']
90 | ['0.075977', '0.050678', '0.162746', '0.358027', '0.311028', '0.050678', '0.445788']
91 | ['0.076295', '0.050890', '0.163426', '0.359525', '0.312329', '0.050890', '0.447652']
92 | ['0.076614', '0.051103', '0.164110', '0.361029', '0.313635', '0.051103', '0.449525']
93 | ['0.076934', '0.051317', '0.164796', '0.362539', '0.314947', '0.051317', '0.451405']
94 | ['0.077256', '0.051531', '0.165486', '0.364055', '0.316264', '0.051531', '0.453293']
95 | ['0.077579', '0.051747', '0.166178', '0.365578', '0.317587', '0.051747', '0.455189']
96 | ['0.077904', '0.051963', '0.166873', '0.367107', '0.318915', '0.051963', '0.457093']
97 | ['0.078229', '0.052181', '0.167571', '0.368642', '0.320249', '0.052181', '0.459004']
98 | ['0.078557', '0.052399', '0.168272', '0.370184', '0.321589', '0.052399', '0.460924']
99 | ['0.078885', '0.052618', '0.168975', '0.371732', '0.322934', '0.052618', '0.462852']
100 | ['0.079215', '0.052838', '0.169682', '0.373287', '0.324284', '0.052838', '0.464788']

⚠ هشدار: همگرایی پس از 100 تکرار حاصل نشد.

نهایی: ['0.079215', '0.052838', '0.169682', '0.373287', '0.324284', '0.052838', '0.464788'] PageRank بردار
بالاترین امتیاز را دارد، همان‌طور که در متن کتاب توضیح داده شده (اینکس 6) d6 نکته: صفحه

نکته:

- را بدون جبر خطی پیچیده محاسبه کرد PageRank این پیاده‌سازی نشان می‌دهد که چگونه می‌توان
- در موتورهای جستجو است PageRank پایهی الگوریتم‌های واقعی محاسبه، Power Iteration روش
- بستگی دارد، نه به نقطه شروع α توزیع ایستا صرفاً به ساختار گراف و مقدار

21.2.3 PageRank شخصی‌سازی‌شده: وقتی گوگل شما را می‌شناسد

🔗 فراتر از PageRank عمومی: دنیای PageRank موضوعی

تا اینجا، PageRank را به‌عنوان معیاری برای اهمیت عمومی یک صفحه وب شناختیم. اما یک سؤال مهم: آیا اهمیت همه صفحات برای همه کاربران یکسان است؟ یک ورزشکار حرفه‌ای به دنبال نتایج زنده فوتبال است و یک محقق سیاسی به دنبال تحلیل‌های روز. آیا باید هر دو نتایج یکسانی دریافت کنند؟

اینجاست که ایده **PageRank موضوعی** (Topic-specific PageRank) وارد می‌شود. در این مدل، ما فرض می‌کنیم کاربران به موضوعات خاصی علاقه دارند. برای مثال، یک کاربر علاقه‌مند به ورزش، ترجیح می‌دهد صفحات مرتبط با ورزش در نتایج جستجویش رتبه بالاتری داشته باشند.

چگونه این کار را انجام می‌دهیم؟ با تغییر قوانین بازی تله‌پورت! به‌جای اینکه Random Surfer ما به یک صفحه وب کاملاً تصادفی پرش کند، او را به یک صفحه وب تصادفی اما مرتبط با موضوع مورد علاقه پرش می‌دهیم.

برای مثال، اگر موضوع مورد نظر "ورزش" باشد، ما مجموعه‌ای از صفحات ورزشی معتبر (مثلاً از طریق دایرکتوری‌هایی مانند DMOZ یا با استفاده از دانش خود موتور جستجو) را شناسایی می‌کنیم. سپس، هر بار که سورفینگ ما تصمیم به تله‌پورت می‌گیرد، او را به یکی از این صفحات ورزشی می‌فرستیم.

از آنجایی که وب‌سایت‌های ورزشی تمایل دارند به یکدیگر لینک دهند (مثلاً ESPN به سایت رسمی یک تیم)، این سورفینگ ورزشی علاقه‌مند به احتمال زیاد در اکوسیستم ورزش باقی می‌ماند. در نتیجه، **توزیع ایستای جدید** که به دست می‌آید (آن را π^* ورزش می‌نامیم)، امتیاز بالاتری به صفحات ورزشی اختصاص می‌دهد.

PageRank شخصی‌سازی‌شده: ترکیب علایق

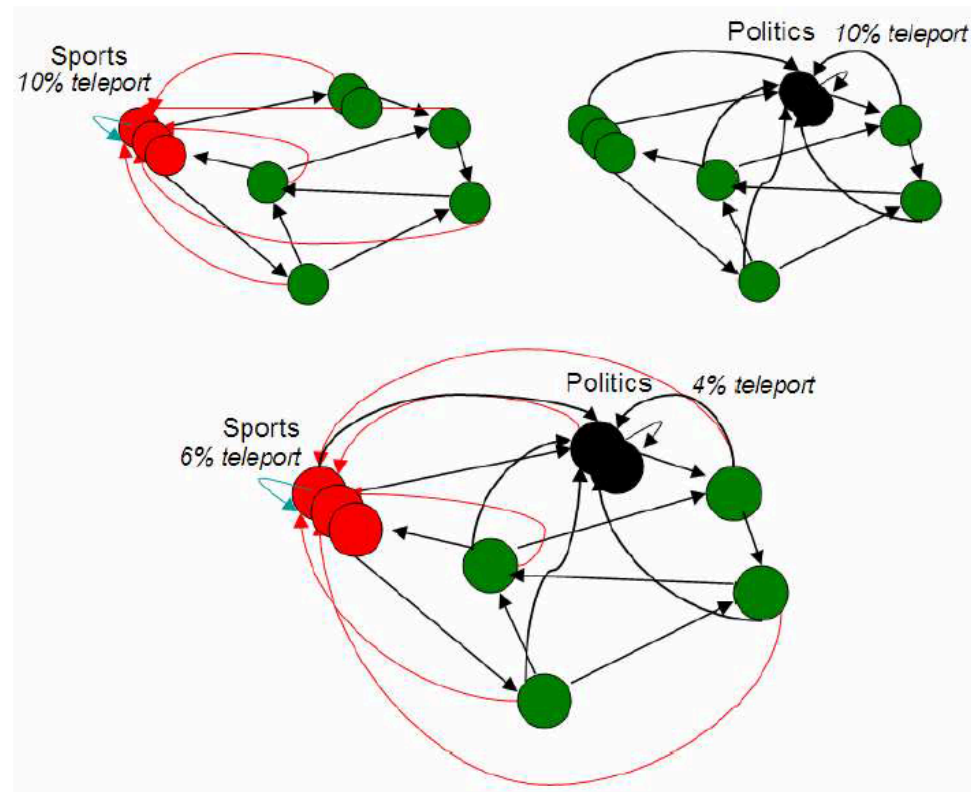
اما افراد معمولاً فقط به یک موضوع علاقه ندارند. یک کاربر ممکن است هم به ورزش و هم به سیاست علاقه داشته باشد. اینجاست که ایده **PageRank شخصی‌سازی‌شده** (Personalized PageRank) به کمک ما می‌آید.

فرض کنید کاربری به نام سارا وجود دارد که علایق او ترکیبی از ورزش (۶۰٪) و سیاست (۴۰٪) است. چگونه می‌توانیم PageRank را برای او شخصی‌سازی کنیم؟

روند سورفینگ سارا به این صورت خواهد بود:

- هر بار که سارا تصمیم به تله‌پورت می‌گیرد، ابتدا سیستم تصمیم می‌گیرد که او را به دنیای ورزش بفرستد یا به دنیای سیاست. این تصمیم با احتمال ۶۰٪ به نفع ورزش و با احتمال ۴۰٪ به نفع سیاست گرفته می‌شود.
- سپس، اگر قرار بود به دنیای ورزش برود، او را به یک صفحه تصادفی از میان مجموعه صفحات ورزشی می‌فرستد. اگر هم قرار بود به دنیای سیاست برود، او را به یک صفحه تصادفی از میان مجموعه صفحات سیاسی می‌فرستد.

این مدل منجر به یک زنجیره مارکوف جدید با توزیع ایستای منحصربه‌فردی می‌شود که دقیقاً متناسب با علایق سارا است.



► Figure 21.5 Topic-specific PageRank. In this example we consider a user whose interests are 60% sports and 40% politics. If the teleportation probability is 10%, this user is modeled as teleporting 6% to sports pages and 4% to politics pages.

شکل 21.5: نمایش PageRank موضوعی. در این مثال، کاربری با علایق ترکیبی (۶۰٪ ورزش و ۴۰٪ سیاست) در نظر گرفته شده است. اگر احتمال تله‌پورت ۱۰٪ باشد، این کاربر به این صورت مدل می‌شود که ۶٪ مواقع به صفحات ورزشی و ۴٪ مواقع به صفحات سیاسی تله‌پورت می‌کند.

⚡ معجزه خطی: محاسبه کارآمد برای میلیون‌ها کاربر

در نگاه اول، ممکن است این ایده بسیار پرهزینه به نظر برسد. آیا باید برای هر کاربر با هر ترکیبی از علایق، یک محاسبه جداگانه انجام دهیم؟ این کار غیرممکن به نظر می‌رسد!

اما اینجاست که جادوی جبر خطی به کمک ما می‌آید. نیازی نیست برای هر کاربر یک PageRank جداگانه محاسبه کنیم. به جای آن، فقط کافی است **بردارهای PageRank موضوعی پایه** را یکبار برای همیشه محاسبه کنیم (یعنی یک PageRank برای ورزش، یک PageRank برای سیاست، و غیره).

سپس، PageRank شخصی‌سازی‌شده برای هر کاربر به سادگی یک **ترکیب خطی** از این بردارهای پایه است. برای سارا (۶۰٪ ورزش و ۴۰٪ سیاست)، PageRank شخصی‌سازی‌شده او به این صورت محاسبه می‌شود:

$$\text{سیاست} \sim \pi + \text{ورزش} \sim \pi 0.4 + \text{سارا} \sim \pi 0.6$$

این ویژگی، استفاده از PageRank شخصی‌سازی‌شده را حتی برای میلیاردها کاربر با پروفایل‌های مختلف ممکن می‌سازد. این روش به موتورهای جستجو اجازه می‌دهد تا نتایجی را ارائه دهند که دقیقاً متناسب با علایق فردی هر کاربر است.

In [5]: # موضوعی PageRank پیاده‌سازی

```
def topic_specific_pagerank(graph, topic_nodes, alpha=0.1, tolerance=1e-6, max_iterations=100):
    """
    (topic_nodes) موضوعی برای یک زیرمجموعه از صفحات PageRank محاسبه.

    ورودی‌ها:
    - graph: دیکشنری گراف وب (صفحه → لیست لینک‌های خروجی)
    - topic_nodes: مجموعه‌ای از صفحات مرتبط با یک موضوع خاص
    - alpha: احتمال تله‌پورت (معمولاً 0.1)
    - tolerance, max_iterations: پارامترهای همگرایی
```

```

خروجی:
غیرصفر است (topic_nodes صفحات قابل‌دسترسی از)  $\gamma$  که فقط برای صفحات در مجموعه PageRank دیکشنری -
"""

# رسید topic_nodes صفحاتی که می‌توان به آن‌ها از  $\gamma$  پیدا کردن تمام گره‌های مرتبط با موضوع
all_nodes = set(graph.keys())
for neighbors in graph.values():
    all_nodes.update(neighbors)

# نگاشت گره‌ها به اندیس
nodes = list(all_nodes)
node_to_index = {node: i for i, node in enumerate(nodes)}
N = len(nodes)

# PageRank مقداردهی اولیه
pagerank = [1.0 / N] * N

# توزیع تله‌پورت: فقط به گره‌های موضوعی
teleport_dist = [0.0] * N
topic_count = 0
for i, node in enumerate(nodes):
    if node in topic_nodes:
        teleport_dist[i] = 1.0
        topic_count += 1

# اگر هیچ گره موضوعی وجود نداشت، تله‌پورت یکنواخت باشد
if topic_count == 0:
    teleport_dist = [1.0 / N] * N
else:
    teleport_dist = [p / topic_count for p in teleport_dist] # توزیع یکنواخت روی موضوع

# روش تکراری
for iteration in range(max_iterations):
    new_pagerank = [0.0] * N

    # توزیع موضوعی * (1 -  $\alpha$ ): بخش تله‌پورت
    for i in range(N):
        new_pagerank[i] += (1.0 - alpha) * teleport_dist[i]

    # بخش انتشار از طریق لینک‌ها:  $\alpha * \sum (PR(P_i) / L(P_i))$ 
    for node in nodes:
        i = node_to_index[node]
        if pagerank[i] == 0:
            continue
        out_links = graph.get(node, [])
        if not out_links:
            continue
        share = (alpha * pagerank[i]) / len(out_links)
        for neighbor in out_links:
            if neighbor in node_to_index:
                j = node_to_index[neighbor]
                new_pagerank[j] += share

    # بررسی همگرایی
    diff = sum(abs(new_pagerank[i] - pagerank[i]) for i in range(N))
    if diff < tolerance:
        break
    pagerank = new_pagerank[:]

# ساخت دیکشنری نتیجه
result = {}
for i, node in enumerate(nodes):
    result[node] = pagerank[i]
return result

def personalized_pagerank(sports_pagerank, politics_pagerank, sports_weight=0.6):
    """
    موضوعی PageRank شخصی‌سازی‌شده با ترکیب خطی دو PageRank ساخت
    """
    all_nodes = set(sports_pagerank.keys()) | set(politics_pagerank.keys())
    personalized = {}
    for node in all_nodes:
        sp = sports_pagerank.get(node, 0.0)
        pp = politics_pagerank.get(node, 0.0)
        personalized[node] = sports_weight * sp + (1 - sports_weight) * pp
    return personalized

# مثال کاربردی: گراف نمونه با دو حوزه موضوعی
if __name__ == "__main__":
    # (سیاسی)  $p_0, p_1, p_2$  و (ورزشی)  $s_0, s_1, s_2$  گراف نمونه: گره‌های
    sample_graph = {
        # (لینک‌های درون‌موضوعی زیاد) صفحات ورزشی
        "s0": ["s1", "s2", "p0"],
        "s1": ["s0", "s2"],
        "s2": ["s0", "s1"],
        # (لینک‌های درون‌موضوعی زیاد) صفحات سیاسی
        "p0": ["p1", "p2", "s0"],
        "p1": ["p0", "p2"],
        "p2": ["p0", "p1"],
        # صفحه عمومی
        "general": ["s0", "p0"]
    }

    # تعریف مجموعه‌های موضوعی
    sports_nodes = {"s0", "s1", "s2"}
    politics_nodes = {"p0", "p1", "p2"}

```

```
print("موضوعی و شخصی‌سازی‌شده PageRank مثال آموزشی")
print("=" * 55)

# موضوعی برای ورزش PageRank محاسبه
sports_pr = topic_specific_pagerank(
    sample_graph,
    sports_nodes,
    alpha=0.1
)

# موضوعی برای سیاست PageRank محاسبه
politics_pr = topic_specific_pagerank(
    sample_graph,
    politics_nodes,
    alpha=0.1
)

# شخصی‌سازی‌شده (60% ورزش، 40% سیاست) PageRank ساخت
personalized_pr = personalized_pagerank(sports_pr, politics_pr, sports_weight=0.6)

# نمایش نتایج
print("\n📊 PageRank (ورزش) - ۱۰ گره برتر")
sorted_sports = sorted(sports_pr.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in sorted_sports:
    if score > 0:
        print(f" {node}: {score:.6f}")

print("\n📊 PageRank (سیاست) - ۱۰ گره برتر")
sorted_politics = sorted(politics_pr.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in sorted_politics:
    if score > 0:
        print(f" {node}: {score:.6f}")

print("\n🧠 PageRank (60% ورزش + 40% سیاست) - ۱۰ گره برتر")
sorted_personal = sorted(personalized_pr.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in sorted_personal:
    print(f" {node}: {score:.6f}")

print("\n💡 نکته:")
print("- PageRank امتیاز بیشتری به صفحات درون یک حوزه اختصاص می‌دهد")
print("- PageRank شخصی‌سازی‌شده با ترکیب خطی ساخته می‌شود - نیازی به محاسبه مجدد نیست")
print("- این روش، امکان سفارشی‌سازی رتبه‌بندی برای میلیون‌ها کاربر را فراهم می‌کند")
```

موضوعی و شخصی‌سازی‌شده PageRank مثال آموزشی
=====

```
📊 PageRank (ورزش) - ۱۰ گره برتر:
s0: 0.333119
s1: 0.327478
s2: 0.327478
p0: 0.011143
p2: 0.000391

📊 PageRank (سیاست) - ۱۰ گره برتر:
p0: 0.333119
p2: 0.327478
p1: 0.327478
s0: 0.011143
s1: 0.000391

🧠 PageRank (60% ورزش + 40% سیاست) - ۱۰ گره برتر:
s0: 0.204329
s1: 0.196643
s2: 0.196643
p0: 0.139934
p1: 0.131226

💡 نکته:
- موضوعی، امتیاز بیشتری به صفحات درون یک حوزه اختصاص می‌دهد.
- PageRank شخصی‌سازی‌شده با ترکیب خطی ساخته می‌شود - نیازی به محاسبه مجدد نیست
- این روش، امکان سفارشی‌سازی رتبه‌بندی برای میلیون‌ها کاربر را فراهم می‌کند.
```

21.3 هاب‌ها و مقام‌ها: کشف گنجینه‌های اطلاعاتی در وب 📖

🔍 سناریوی واقعی: جستجوی اطلاعات پزشکی

تصور کنید به تازگی به بیماری "میگرن" مبتلا شده‌اید و می‌خواهید اطلاعات معتبر و قابل اعتماد درباره آن پیدا کنید. اگر عبارت "میگرن" را در یک موتور جستجو وارد کنید، با هزاران نتیجه مواجه می‌شوید. کدام صفحات معتبر هستند و کدام صرفاً ادعا می‌کنند که متخصص هستند؟

در اینجا، دو نوع صفحه وب وجود دارد که برای شما مفید خواهند بود:

1. **صفحات مقام (Authority):** این صفحات منابع اصلی و معتبر اطلاعات هستند. در مورد میگرد، این صفحات شامل وبسایت‌های انجمن‌های پزشکی معتبر، مقالات علمی در مجلات پزشکی، یا صفحات وبسایت‌های سازمان‌های بهداشتی معتبر است.

2. **صفحات هاب (Hub):** این صفحات خودشان حاوی اطلاعات اصلی نیستند، بلکه مجموعه‌ای از لینک‌ها به منابع معتبر را گردآوری کرده‌اند. مثلاً یک وبلاگ که توسط یک بیمار میگردی اداره می‌شود و بهترین منابع اطلاعاتی درباره میگرد را معرفی کرده است.

الگوریتم **HITS** (Hyperlink-Induced Topic Search) دقیقاً برای همین منظور طراحی شده است: برای هر پرس‌وجو، دو امتیاز جداگانه به هر صفحه اختصاص می‌دهد:

- **امتیاز هاب (Hub Score):** نشان‌دهنده توانایی صفحه در اشاره به صفحات معتبر است.
- **امتیاز مقام (Authority Score):** نشان‌دهنده اعتبار صفحه به‌عنوان منبع اطلاعات است.

🔗 رابطه متقابل هاب‌ها و مقام‌ها

ایده کلیدی پشت الگوریتم HITS این است که هاب‌ها و مقام‌ها به‌صورت متقابل یکدیگر را تقویت می‌کنند:

- یک **هاب خوب** صفحه‌ای است که به مقام‌های خوب لینک می‌دهد.
- یک **مقام خوب** صفحه‌ای است که توسط هاب‌های خوب به آن لینک داده شده است.

این رابطه متقابل را می‌توان با معادلات زیر نشان داد:

$$\begin{aligned}h(v) &\leftarrow \sum_{y \rightarrow v} a(y) \\a(v) &\leftarrow \sum_{y \rightarrow v} h(y)\end{aligned}$$

که در آن: $h(v)$ امتیاز هاب صفحه v است، $a(v)$ امتیاز مقام صفحه v است. - مجموع اول برابر است با مجموع امتیازات مقام صفحاتی که v به آن‌ها لینک داده است. - مجموع دوم برابر است با مجموع امتیازات هاب صفحاتی که به v لینک داده‌اند.

این معادلات به‌صورت ماتریسی بازنویسی می‌شوند:

$$\begin{aligned}\sim h &\leftarrow A \sim a \\ \sim a &\leftarrow A^T \sim h\end{aligned}$$

که در آن A ماتریس مجاورت گراف وب است. با جایگزینی مکرر، به معادلات ویژه‌مقداری می‌رسیم:

$$\begin{aligned}\sim h &= (1/\lambda_h) A A^T \sim h \\ \sim a &= (1/\lambda_a) A^T A \sim a\end{aligned}$$

بنابراین، بردارهای نهایی هاب و مقام، **بردارهای ویژه اصلی** ماتریس‌های $A A^T$ و $A^T A$ هستند. در عمل، این بردارها با روش‌های تکراری (مثل Power Iteration) محاسبه می‌شوند.

🔗 مثال کاربردی: جستجوی اطلاعات در مورد هوش مصنوعی

فرض کنید می‌خواهید اطلاعات معتبر در مورد "یادگیری عمیق" پیدا کنید. در این حالت:

- **مقام‌ها** ممکن است شامل صفحات وبسایت‌های دانشگاهی معتبر (مثل استنفورد، ام‌آی‌تی)، مقالات بنیادی در این زمینه، یا مستندات رسمی فریمورک‌های معتبر یادگیری عمیق (مثل TensorFlow یا PyTorch) باشند.
 - **هاب‌ها** ممکن است شامل وبلاگ‌های تخصصی که بهترین منابع یادگیری عمیق را معرفی کرده‌اند، یا صفحه‌هایی که مجموعه‌ای از مقالات مهم و آموزش‌های باکیفیت را گردآوری کرده‌اند، باشند.
- الگوریتم HITS با تحلیل لینک‌ها بین این صفحات، می‌تواند به شما کمک کند تا هم منابع اصلی و معتبر (مقام‌ها) و هم راهنماهای مفید برای پیدا کردن این منابع (هاب‌ها) را شناسایی کنید.

🇮🇷 مراحل اجرای الگوریتم HITS

الگوریتم HITS به صورت زیر عمل می‌کند:

1. برای پرس‌وجوی داده‌شده، یک زیرمجموعه از صفحات وب مرتبط را جمع‌آوری می‌کنیم (این فرآیند در بخش بعدی توضیح داده می‌شود).
2. برای هر صفحه در این زیرمجموعه، امتیاز هاب و مقام اولیه را 1 در نظر می‌گیریم.
3. به صورت تکراری، امتیازات هاب و مقام را با استفاده از معادلات به‌روزرسانی می‌کنیم تا زمانی که مقادیر همگرا شوند.
4. در نهایت، صفحات را بر اساس امتیازات هاب و مقام رتبه‌بندی کرده و دو لیست جداگانه ارائه می‌دهیم.

نکته کلیدی این است که HITS فقط ساختار لینک‌ها را تحلیل می‌کند و به محتوای متن صفحات توجهی ندارد (مگر در مرحله جمع‌آوری زیرمجموعه اولیه).

📊 مثال عددی: تحلیل یک شبکه کوچک

در مثال 21.2 برای پرس‌وجوی "jaguar" (که هم می‌تواند به حیوان و هم به خودرو اشاره کند)، الگوریتم HITS صفحات زیر را به عنوان مقام‌ها و هاب‌های اصلی شناسایی می‌کند:

- صفحه q3 به عنوان **مقام اصلی** شناخته می‌شود، زیرا دو هاب (q2 و q6) از طریق لینک‌هایی با متن "jaguar" به آن اشاره کرده‌اند.
- صفحات q2 و q6 به عنوان **هاب‌های اصلی** شناخته می‌شوند، زیرا به مقام‌های معتبر لینک داده‌اند.

این مثال نشان می‌دهد که چگونه HITS می‌تواند با تحلیل ساختار لینک‌ها، صفحات معتبر را از صفحات کم‌اهمیت‌تر تمیز دهد، حتی وقتی کلمات کلیدی به تنهایی کافی نیستند.

In [6]: *# HITS (Hubs and Authorities) پیاده‌سازی الگوریتم*

```
def hits_algorithm(graph, max_iterations=100, tolerance=1e-6):
    """
    محاسبه امتیازهای هاب و مقام برای گراف داده‌شده.

    ورودی‌ها:
    - graph: دیکشنری نمایش‌دهنده گراف جهت‌دار
      مثال: {"A": ["B", "C"], "B": ["C"], "C": ["A"]}
    - max_iterations: حداکثر تعداد تکرار
    - tolerance: آستانه همگرایی

    خروجی:
    - دیکشنری شامل دو کلید:
      'hubs': دیکشنری امتیاز هاب‌ها
      'authorities': دیکشنری امتیاز مقام‌ها
    """
    # جمع‌آوری تمام گره‌ها
    all_nodes = set(graph.keys())
    for neighbors in graph.values():
        all_nodes.update(neighbors)
    nodes = sorted(all_nodes) # مرتب‌سازی برای نتایج قابل‌پیش‌بینی

    # مقداردهی اولیه: همه امتیازها برابر 1
    hubs = {node: 1.0 for node in nodes}
    authorities = {node: 1.0 for node in nodes}

    for iteration in range(max_iterations):
        # مرحله 1: به‌روزرسانی امتیاز مقام‌ها
        new_authorities = {}
        for node in nodes:
            auth_score = 0.0
            # جمع امتیاز هاب‌هایی که به این گره لینک داده‌اند
            for source, targets in graph.items():
                if node in targets:
                    auth_score += hubs.get(source, 0.0)
            new_authorities[node] = auth_score

        # مرحله 2: به‌روزرسانی امتیاز هاب‌ها
        new_hubs = {}
        for node in nodes:
            hub_score = 0.0
            # جمع امتیاز مقام‌هایی که این گره به آنها لینک داده
            for target in graph.get(node, []):
                hub_score += new_authorities.get(target, 0.0)
            new_hubs[node] = hub_score

        # نرمال‌سازی برای جلوگیری از انفجار عددی
        # HITS رایج‌ترین روش در L2 نرمال‌سازی
        def normalize(score_dict):
            total_sq = sum(v**2 for v in score_dict.values())
            if total_sq == 0:
                return {k: 0.0 for k in score_dict}
            norm = total_sq ** 0.5
            return {k: v / norm for k, v in score_dict.items()}

        new_authorities = normalize(new_authorities)
        new_hubs = normalize(new_hubs)

        # بررسی همگرایی
        auth_diff = sum(abs(new_authorities[n] - authorities[n]) for n in nodes)
        hub_diff = sum(abs(new_hubs[n] - hubs[n]) for n in nodes)

        if auth_diff < tolerance and hub_diff < tolerance:
            print(f"✅ تکرار {iteration + 1} همگرایی پس از")
            break

        authorities = new_authorities
        hubs = new_hubs

    return {"hubs": hubs, "authorities": authorities}

# مثال کاربردی: گراف شکل 21.4 (ساده‌شده برای آموزش)
if __name__ == "__main__":
    # d0 تا d6 گراف نمونه مبتنی بر شکل 21.4 با گره‌های
    sample_graph = {
        "d0": ["d2", "d1"],
        "d1": ["d2"],
        "d2": ["d0", "d3"],
        "d3": ["d4", "d6"],
        "d4": ["d6"],
        "d5": ["d6"],
        "d6": ["d3"]
    }

    print("برای گراف شکل 21.4 HITS مثال آموزشی الگوریتم")
    print("=" * 50)

    # اجرای الگوریتم HITS
    results = hits_algorithm(sample_graph)
    hubs = results["hubs"]
    authorities = results["authorities"]

    # نمایش ۳ گره برتر در هر دسته
    print("\n 🏆 (Authorities): گره برتر از نظر امتیاز مقام ۳")
    top_authorities = sorted(authorities.items(), key=lambda x: x[1], reverse=True)[:3]
```

```
for rank, (node, score) in enumerate(top_authorities, 1):
    print(f" {rank}. {node}: {score:.6f}")

print("\n🌐 ۳ گره برتر از نظر امتیاز هاب (Hubs):")
top_hubs = sorted(hubs.items(), key=lambda x: x[1], reverse=True)[:3]
for rank, (node, score) in enumerate(top_hubs, 1):
    print(f" {rank}. {node}: {score:.6f}")

print("\n💡 نکته:")
print("- دو نوع صفحه را شناسایی می‌کند HITS الگوریتم -")
print("مقام‌ها: صفحاتی که توسط هاب‌های متعددی لینک شده‌اند •")
print("هاب‌ها: صفحاتی که به مقام‌های متعددی لینک داده‌اند •")
print("- وابسته به پرس‌وجو است و روی یک زیرگراف موضوعی اجرا می‌شود HITS PageRank برخلاف -")
print("- جلوی رشد نامحدود امتیازها را می‌گیرد و همگرایی را تضمین می‌کند L2 نرمال‌سازی -")
```

برای گراف شکل HITS 21.4 مثال آموزشی الگوریتم

=====

✓ همگرایی پس از 52 تکرار

🏆 (Authorities) گره برتر از نظر امتیاز مقام ۳:

1. d6: 0.923880
2. d4: 0.382683
3. d2: 0.000001

🌐 (Hubs) گره برتر از نظر امتیاز هاب ۳:

1. d3: 0.707107
2. d4: 0.500000
3. d5: 0.500000

💡 نکته:

- دو نوع صفحه را شناسایی می‌کند HITS الگوریتم
- مقام‌ها: صفحاتی که توسط هاب‌های متعددی لینک شده‌اند
- هاب‌ها: صفحاتی که به مقام‌های متعددی لینک داده‌اند
- وابسته به پرس‌وجو است و روی یک زیرگراف موضوعی اجرا می‌شود HITS PageRank برخلاف
- جلوی رشد نامحدود امتیازها را می‌گیرد و همگرایی را تضمین می‌کند L2 نرمال‌سازی

21.3.1 انتخاب زیرمجموعه وب: هنر پیدا کردن گنجینه‌های پنهان 📖

🔍 چالش اصلی: صفحات معتبر بدون کلمات کلیدی

یک چالش بزرگ در الگوریتم HITS این است که بسیاری از صفحات معتبر ممکن است کلمه کلیدی پرس‌وجو را در محتوای خود نداشته باشند. این پدیده به خصوص برای شرکت‌های بزرگ و برندهای معتبر رایج است.

برای مثال، وب‌سایت شرکت اپل منبع معتبری برای اطلاعات درباره آیفون است، اما ممکن است صفحه اصلی آن کلمه "آیفون" را نداشته باشد و فقط از برند "Apple" استفاده کند. یا وب‌سایت سازمان بهداشت جهانی (WHO) منبع معتبری برای اطلاعات درباره کرونا است، اما ممکن است در صفحه اصلی خود از کلمه "کرونا" استفاده نکند و عبارت "COVID-19" را به کار ببرد.

این چالش باعث می‌شود که اگر فقط صفحاتی را که شامل کلمه کلیدی پرس‌وجو هستند در نظر بگیریم، ممکن است منابع معتبر و اصلی را از دست بدهیم.

🚀 راه‌حل: از مجموعه ریشه تا مجموعه پایه

برای حل این مشکل، از یک فرآیند دو مرحله‌ای برای ساخت مجموعه پایه استفاده می‌کنیم:

1. **مجموعه ریشه (Root Set):** تمام صفحاتی که کلمه کلیدی پرس‌وجو را در متن خود دارند. برای مثال،

اگر پرس‌وجوی ما "هوش مصنوعی در پزشکی" باشد، مجموعه ریشه شامل تمام صفحاتی است که این عبارت را در متن خود دارند.

2. **مجموعه پایه (Base Set):** شامل مجموعه ریشه به‌علاوه تمام صفحاتی که یا به صفحات ریشه لینک

داده‌اند یا توسط صفحات ریشه لینک شده‌اند.

برای درک بهتر، این مثال را در نظر بگیرید: فرض کنید در حال جستجوی اطلاعات درباره "سرطان سینه" هستیم. مجموعه ریشه شامل صفحاتی است که مستقیماً از این عبارت استفاده می‌کنند. اما وبسایت انجمن سرطان آمریکا، که یک منبع بسیار معتبر است، ممکن است در صفحه اصلی خود از عبارت "سرطان سینه" استفاده نکند و فقط از عبارت "Breast Cancer" استفاده کند.

با این حال، اگر یک وبلاگ پزشکی (که در مجموعه ریشه قرار دارد) به وبسایت انجمن سرطان آمریکا لینک داده باشد، این وبسایت معتبر وارد مجموعه پایه می‌شود و در نتیجه در تحلیل HITS شرکت داده خواهد شد.

💡 مزایای کلیدی این رویکرد

این روش گسترش مجموعه ریشه به مجموعه پایه سه مزیت کلیدی دارد:

- **شناسایی مقام‌های پنهان:** صفحات معتبری که کلمه کلیدی را ندارند، از طریق لینک‌های ورودی از هاب‌های مرتبط شناسایی می‌شوند.
- **گسترش شبکه مقام‌ها:** اگر یک هاب معتبر در مجموعه ریشه باشد، تمام مقام‌هایی که به آن‌ها لینک داده، در مجموعه پایه گنجانده می‌شوند.
- **شناسایی هاب‌های مرتبط:** اگر یک مقام معتبر در مجموعه ریشه باشد، تمام هاب‌هایی که به آن لینک داده‌اند، وارد مجموعه پایه می‌شوند.

به طور خلاصه، این گسترش به ما کمک می‌کند تا یک شبکه غنی از هاب‌ها و مقام‌های مرتبط با موضوع مورد نظر ایجاد کنیم، حتی اگر برخی از این صفحات مستقیماً کلمه کلیدی پرس‌وجو را در متن خود نداشته باشند.

🌐 پدیده شگفت‌انگیز: بازیابی بین‌زبانی

یکی از جالب‌ترین نتایج الگوریتم HITS این است که می‌تواند صفحات را به زبان‌های مختلف بازیابی کند، حتی اگر پرس‌وجو به یک زبان خاص باشد. این پدیده که **بازیابی بین‌زبانی** (Cross-Language Retrieval) نامیده می‌شود، کاملاً از طریق تحلیل لینک (نه ترجمه زبانی) حاصل می‌شود.

برای مثال، اگر شما عبارت "japan elementary schools" را به انگلیسی جستجو کنید، ممکن است صفحات ژاپنی (به زبان ژاپنی) در نتایج ظاهر شوند. چطور این اتفاق می‌افتد؟

تصور کنید یک وبلاگ انگلیسی‌زبان درباره سیستم آموزشی ژاپن وجود دارد که به وبسایت‌های مدارس ژاپنی (به زبان ژاپنی) لینک داده است. این وبلاگ انگلیسی در مجموعه ریشه قرار می‌گیرد (چون حاوی کلمات کلیدی انگلیسی است)، اما وبسایت‌های ژاپنی که به آن‌ها لینک داده شده، در مجموعه پایه قرار می‌گیرند. سپس الگوریتم HITS این وبسایت‌های ژاپنی را به‌عنوان مقام‌های معتبر شناسایی می‌کند، حتی اگر به زبان انگلیسی نباشند.

Hubs

- schools
- LINK Page-13
- "ú-[iŠw=Z
- =a%=-Šw=Zfz=[f=fy=[fW
- 100 Schools Home Pages (English)
- K-12 from Japan 10/...met and Education)
- http://www...iglobe.ne.jp/~IKESAN
- ,l,f,j=-Šw=Z,U"N,P'g•œê
- =ÔŠ—'—§=ÔŠ—œ=-Šw=Z
- Koulutus ja oppilaitokset
- TOYODA HOMEPAGE
- Education
- Cay's Homepage(Japanese)
- -y"i=-Šw=Z,i,fz=[f=fy=[fW
- UNIVERSITY
- %œ—³=-Šw=Z DRAGON97-TOP
- =Â%=-Šw=Z,T"N,P'g,fz=[f=fy=[fW
- ¶µ°ê¼,ÂÂ© ¥áE¥áj¼ ¥áE¥áj¼

Authorities

- The American School in Japan
- The Link Page
- %°=-è=-s—§"ā"=-Šw=Zfz=[f=fy=[fW
- Kids' Space
- "À=-é=-s—§"À=-é=-¼•=-Šw=Z
- «{=-é"ç'âŠw=-@=-Šw=Z
- KEIMEI GAKUEN Home Page (Japanese)
- Shiranuma Home Page
- fuzoku-es.fukui-u.ac.jp
- welcome to Miasa E&J school
- =_="p=iœ§=E%oj•=-s—§'†=-i=-¼=-Šw=Z,i,fy
- http://www...p/~m_maru/index.html
- fukui haruyama-es HomePage
- Torisu primary school
- goo
- Yakumo Elementary,Hokkaido,Japan
- FUZOKU Home Page
- Kamishibun Elementary School...

► **Figure 21.6** A sample run of HITS on the query japan elementary schools.

شکل 21.6: نتایج اجرای HITS برای پرس‌وجوی "japan elementary schools". این شکل بالاترین هاب‌ها و مقام‌ها را نشان می‌دهد. هر ردیف عنوان صفحه HTML مربوطه را نمایش می‌دهد. توجه کنید که بسیاری از نتایج به زبان ژاپنی هستند، با وجود اینکه پرس‌وجو به انگلیسی بوده است.

🔧 نکات اجرایی و بهینه‌سازی

در پیاده‌سازی عملی الگوریتم HITS، چند نکته مهم برای بهینه‌سازی عملکرد وجود دارد:

- **اندازه مجموعه ریشه:** نیازی به شامل کردن همه صفحات مرتبط با پرس‌وجو نیست. معمولاً حدود 200 صفحه برای مجموعه ریشه کافی است تا نتایج باکیفیتی تولید شود.
- **تعداد تکرارها:** برای محاسبه امتیازهای هاب و مقام، نیازی به دقت بالا نیست. معمولاً 5 تا 10 تکرار از معادلات HITS برای یافتن رتبه‌بندی نسبی کافی است.
- **بهینه‌سازی محاسبات:** به دلیل پراکنده‌بودن گراف وب (هر صفحه به طور متوسط به حدود 10 صفحه دیگر لینک می‌دهد)، به‌روزرسانی‌ها به‌صورت جمع‌های افزایشی (نه ضرب ماتریسی) انجام می‌شود تا محاسبات کارآمدتر شوند.

این بهینه‌سازی‌ها باعث می‌شوند که الگوریتم HITS بتواند به سرعت و با کارایی بالا نتایج باکیفیتی تولید کند، حتی برای مجموعه‌های بزرگ از صفحات وب.

🔗 جمع‌بندی فصل ۲۱: تحلیل پیوندها

چگونه می‌توان اهمیت و اعتبار یک صفحه وب را بر اساس ساختار پیوندهای آن در وب سنجید؟ فصل ۲۱ به ما نشان می‌دهد که چگونه تحلیل گراف وب (شبکه‌ای از صفحات و پیوندها) منجر به توسعه الگوریتم‌های رتبه‌بندی انقلابی مانند PageRank و HITS شده است. این الگوریتم‌ها با تفسیر هر پیوند به عنوان یک «رأی اعتماد»، توانستند

کیفیت صفحات را به شیوه‌ای فراتر از تطبیق متن ساده ارزیابی کنند و به موتورهای جستجو کمک کنند تا نتایج مرتبط‌تر و معتبرتری را به کاربران ارائه دهند.

- با ****مفهوم متن لنگر (Anchor Text)**** آشنا شدیم. این متن، همان متن قابل کلیک است که در یک پیوند hyperlink قرار دارد و اغلب توصیف دقیقی از صفحه مقصد ارائه می‌دهد. این ویژگی به ویژه برای ایندکس‌گذاری صفحاتی که خودشان به درستی توصیف نشده‌اند (مانند صفحه اصلی IBM که ممکن است کلمه "computer" را نداشته باشد) بسیار مفید است.
- الگوریتم ****PageRank**** را بررسی کردیم. این الگوریتم، که توسط گوگل توسعه یافته، هر صفحه را بر اساس تعداد و کیفیت پیوندهایی که به آن داده شده است، امتیازی بین ۰ و ۱ اختصاص می‌دهد. این مدل بر اساس مفهوم «کاربر تصادفی» (Random Surfer) عمل می‌کند که در وب به صورت تصادفی پیوندها را دنبال می‌کند یا به صورت تصادفی به صفحه‌ای دیگر «تلیپورت» می‌کند. PageRank، احتمال حضور کاربر در یک صفحه در حالت پایدار را نشان می‌دهد و یک معیار کیفیت ایستا و مستقل از پرس‌وجو است.
- با ****PageRank موضوع-محور (Topic-Specific PageRank)**** آشنا شدیم. در این رویکرد، به جای تلیپورت کردن کاربر به یک صفحه کاملاً تصادفی، او به صفحات مرتبط با یک موضوع خاص (مانند ورزش یا سیاست) تلیپورت می‌کند. این کار باعث می‌شود صفحات مرتبط با آن موضوع، امتیاز PageRank بالاتری کسب کنند. این ایده اساس ****PageRank شخصی‌سازی‌شده (Personalized PageRank)**** را نیز فراهم می‌کند که می‌تواند بر اساس علایق ترکیبی یک کاربر، امتیازهای PageRank مختلف را با هم ترکیب کند.
- الگوریتم ****HITS (Hyperlink-Induced Topic Search)**** را مطالعه کردیم. این الگوریتم برای هر صفحه دو امتیاز محاسبه می‌کند: ****امتیاز هاب (Hub Score)**** و ****امتیاز اتوریتی (Authority Score)****. صفحات ****اتوریتی**** منابع معتبر و اصلی در یک موضوع هستند (مانند صفحه یک مؤسسه تحقیقاتی معتبر). صفحات ****هاب**** صفحاتی هستند که به منابع معتبر زیادی لینک می‌دهند (مانند یک فهرست دست‌نویس شده از منابع مفید). این الگوریتم به صورت تکراری این دو امتیاز را محاسبه می‌کند تا بهترین هاب‌ها و اتوریتی‌ها را برای یک پرس‌وجوی مشخص پیدا کند.

بیایید این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

 **قدرت متن لنگر:** ممکن است صفحه اصلی شرکت IBM (ibm.com) کلمه "computer" را در متن خود نداشته باشد. اما هزاران وب‌سایت دیگر در هنگام لینک دادن به IBM، از کلمه "computer" در متن لنگر خود استفاده می‌کنند. موتورهای جستجو از این سیگنال استفاده می‌کنند و به صفحه IBM امتیاز بالایی برای جستجوی "computer" می‌دهند. به همین دلیل، جستجوی "big blue" (لقب معروف IBM) نیز معمولاً صفحه اصلی این شرکت را به عنوان نتیجه برتر نشان می‌دهد.

 **PageRank به مثابه رأی اعتماد:** تصور کنید هر پیوند به یک صفحه، یک رأی اعتماد به آن صفحه است. اما همه رأی‌ها برابر نیستند. یک پیوند از یک وب‌سایت بسیار معتبر و محبوب (مانند یک خبرگزاری بزرگ) بسیار ارزشمندتر از یک پیوند از یک وبلاگ ناشناس است. PageRank دقیقاً همین منطق را پیاده‌سازی می‌کند؛ صفحاتی که از منابع معتبرتری لینک دریافت کرده‌اند، رتبه بالاتری خواهند داشت، زیرا کاربر تصادفی احتمال بیشتری دارد که از طریق آن پیوندهای معتبر به این صفحات برسد.

 **HITS: پیدا کردن مراجع و راهنماها:** اگر شما عبارت "لوکیمی" را جستجو کنید، الگوریتم HITS تلاش می‌کند دو نوع صفحه را پیدا کند: (۱) صفحات ****اتوریته**** مانند وب‌سایت موسسه ملی سرطان که منبع اصلی و معتبر اطلاعات در این زمینه است. (۲) صفحات ****هاب**** مانند وبلاگ یک پزشک یا صفحه‌ای که یک بیمار برای جمع‌آوری منابع مفید درباره لوکیمی ساخته است و به منابع معتبر متعددی (از جمله موسسه ملی سرطان) لینک داده است. HITS با تحلیل این پیوندها، بهترین منابع (اتوریته‌ها) و بهترین راهنماها (هاب‌ها) را شناسایی می‌کند.

 **PageRank شخصی‌سازی‌شده:** یک کاربر ممکن است ۶۰٪ به ورزش و ۴۰٪ به سیاست علاقه داشته باشد. موتور جستجو می‌تواند با ترکیب PageRank موضوع-محور ورزش (که در آن تلپورت به صفحات ورزشی است) و PageRank موضوع-محور سیاست، یک رتبه‌بندی شخصی‌سازی‌شده برای این کاربر ایجاد کند. در این رتبه‌بندی، صفحات مرتبط با ورزش و سیاست بر اساس ترکیب علایق او، امتیاز بالاتری کسب می‌کنند و نتایج جستجوی شخصی‌تری به او نمایش داده می‌شود.

این فصل نشان داد که تحلیل پیوندها با تبدیل وب به یک گراف بزرگ و استفاده از هوش جمعی سازندگان صفحات وب (از طریق پیوندهایشان)، انقلابی در فناوری جستجو ایجاد کرد. الگوریتم‌هایی مانند PageRank و HITS با فراتر رفتن از تطبیق ساده متن، توانستند اعتبار و کیفیت صفحات را بسنجند و این یکی از دلایل اصلی موفقیت موتورهای جستجوی مدرن در ارائه نتایج باکیفیت و مرتبط است.